# TOWARDS A UNIVERSAL FEATURE EXTRACTOR FOR ATARI GAMES

## Reconstruction, FSQ and Joint-Embedding Prediction

Sajid Ali Sadruddin

Technische Universität Dresden (TUD)

September 2026



# Outline

1. Introduction
2. Related Work
3. Methodology
4. Results
5. Conclusion

# Motivation and Research Questions

- Every Atari game gives the agent the same input: 4 stacked $84 \times 84$ grey-scale frames
- Still, each agent trains its own CNN (NatureCNN) from scratch, using only the reward
- Many games look alike: sprites, balls, walls and scrolling backgrounds
- **Idea:** pre-train **one** encoder on many games and reuse it for PPO on every game
- Two design choices: the **objective** (reconstruct the pixels or predict the next latent) and the **usage** (frozen or fine-tuned)

## Research questions

**RQ1** Which objective gives the most useful features for PPO?
**RQ2** Is it better to freeze or to fine-tune the encoder?
**RQ3** Do the features work on games not seen in pre-training?
**RQ4** What is the cost in GPU-hours and energy (kWh), and can the features be inspected, e.g. decoded back into images?

# Related Work: Same Problem, One Model for Many Games

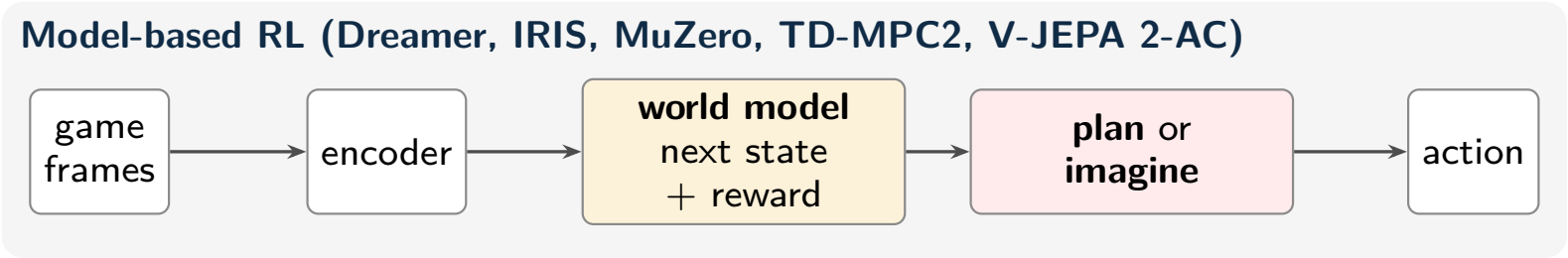
| Approach | What they do | This work |
|---|---|---|
| **One agent, many games**<br>PopArt, Multi-Game DT, Scaled QL | one network plays 40–57 games, trained with rewards | only the encoder is shared, no rewards |
| **Learning from experts**<br>Policy Distillation, Actor-Mimic | one network copies the experts of several games | experts only provide the data |
| **Transfer to a new game**<br>Progressive Nets, Farebrother et al. | reuse the features of one game for the next | one encoder from 11 games at once |

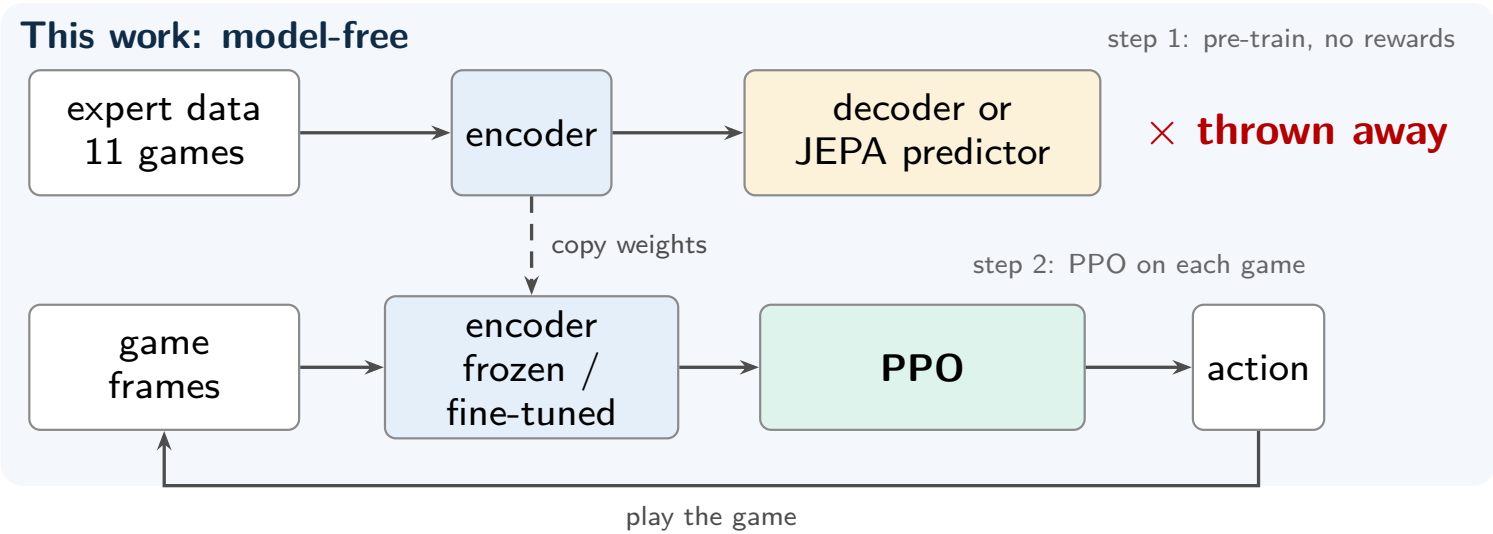
# Related Work: From Images and Videos to Games

| Method | Built for | Changed in this work |
|---|---|---|
| **VAE, VQ-VAE, FSQ** | generating images, video and audio | encoder reused as the feature extractor for PPO; loss on moving pixels |
| **I-JEPA, V-JEPA** | images and videos: predict masked regions | predict the *next* frame stack, given the action and the game; one model for 11 games |
| **TD-MPC2** | one world model for 80 robot-control tasks, used to plan | task embedding and SimNorm reused; no planning |

Key difference: in games, the agent's **actions** change what happens next

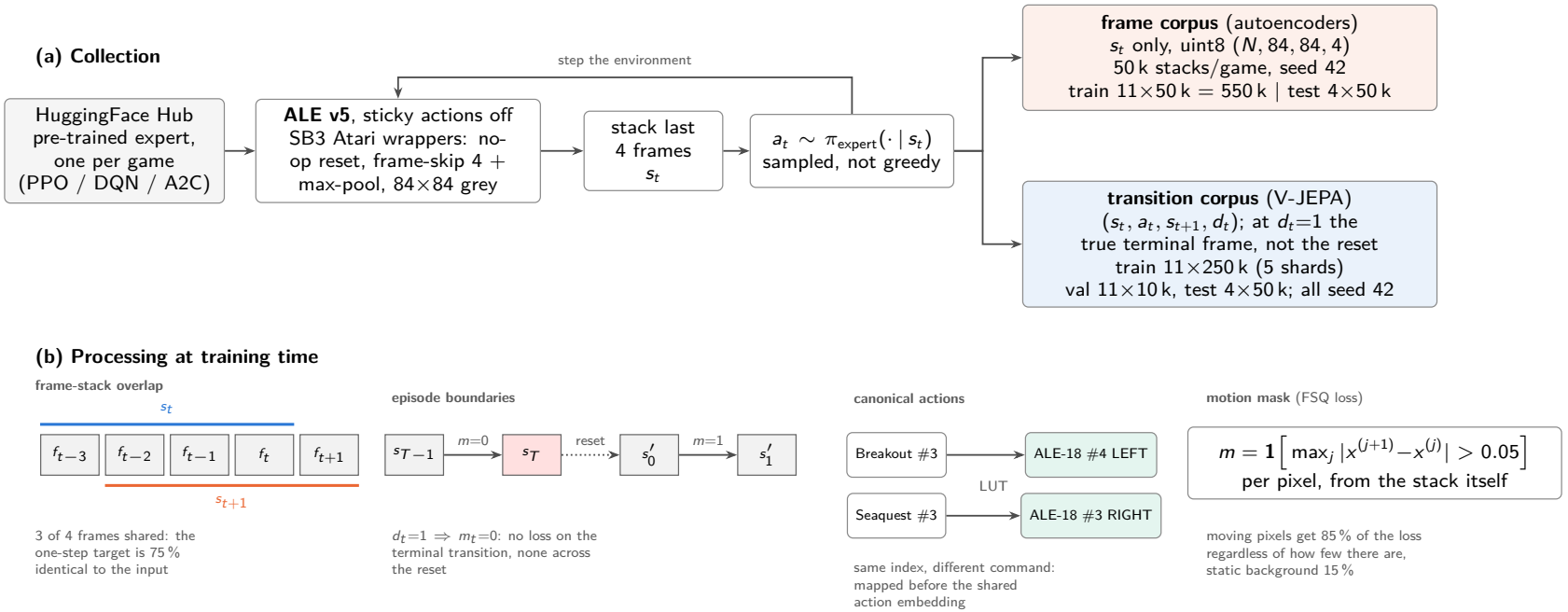
# Related Work: Model-Based vs. Model-Free RL



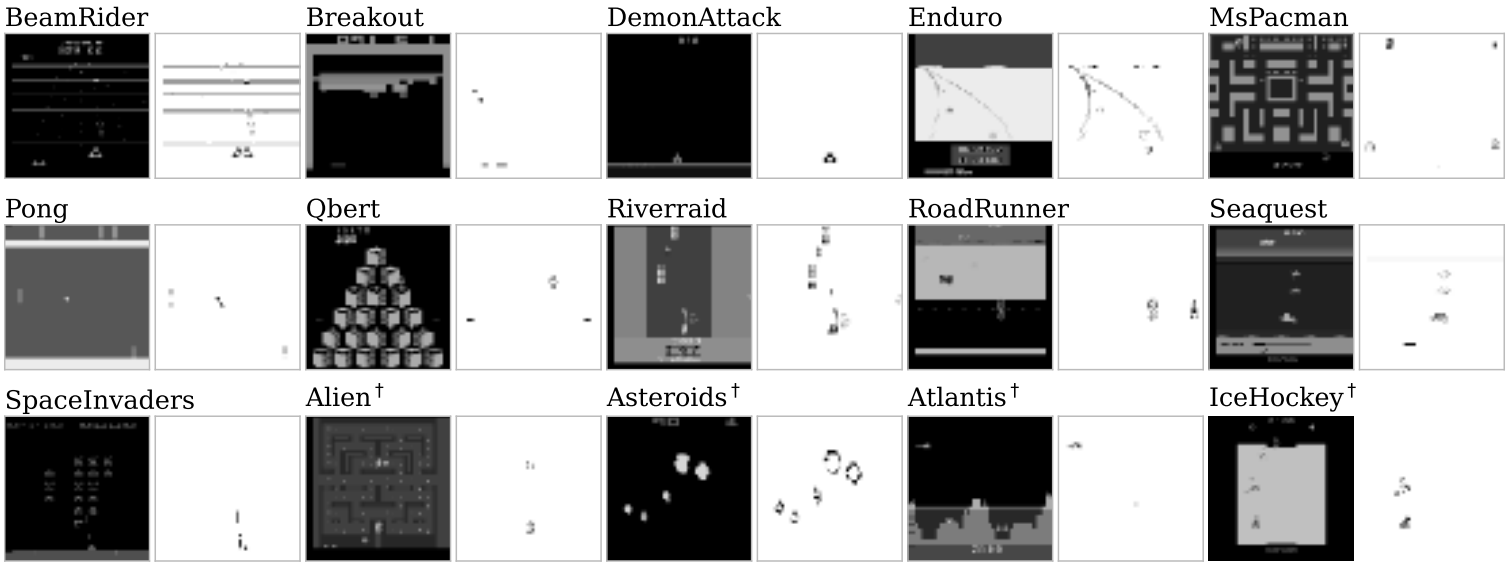

- **Model-based:** the world model is used to choose the actions
- **This work:** the world model only trains the encoder; PPO chooses the actions

# Data Collection




- Random play rarely leaves the first screen, so expert agents from HuggingFace were used
- Actions were sampled from the expert policy, so some play is sub-optimal
- 11 games for pre-training, 4 games held out for testing

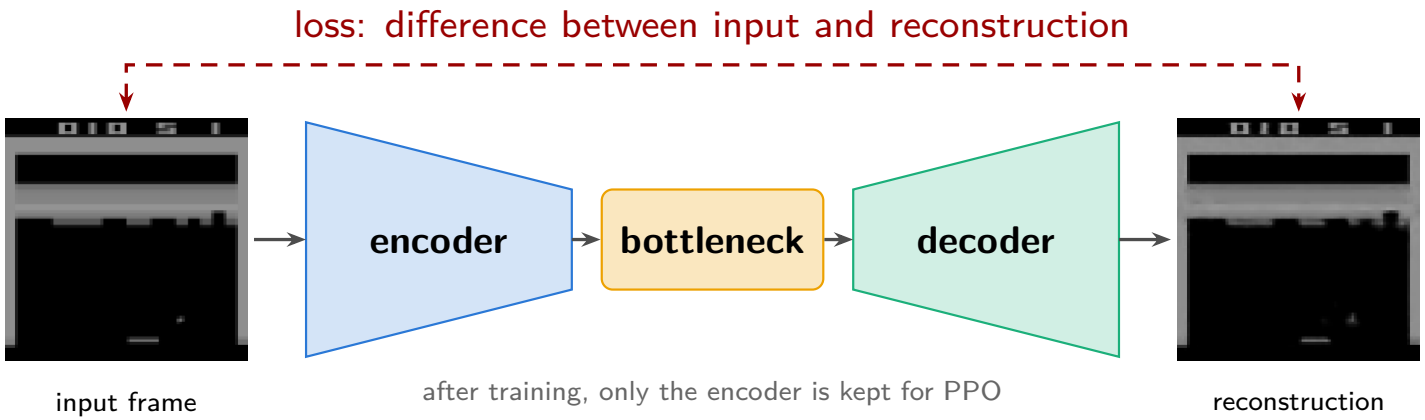
# The Atari Games




Frame $t$ (left) and the change to the next frame (right); † held-out games

- Same input shape, very different content: from 1% to 95% black pixels
- In most games less than 4% of the pixels move: what matters are small sprites
- 4 to 18 actions, and the same action index means a different button in each game

# Autoencoders

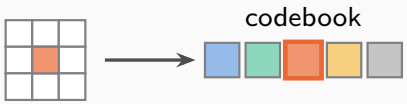



**Standard VAE**

$\mu$ $\sigma$ $\rightarrow z \sim \mathcal{N}(\mu, \sigma^2)$

one vector of 256 numbers
(too coarse for small sprites)

**Residual VQ-VAE**




$21 \times 21$ grid; each cell is replaced by the nearest of 512 code vectors

**FSQ-VAE**

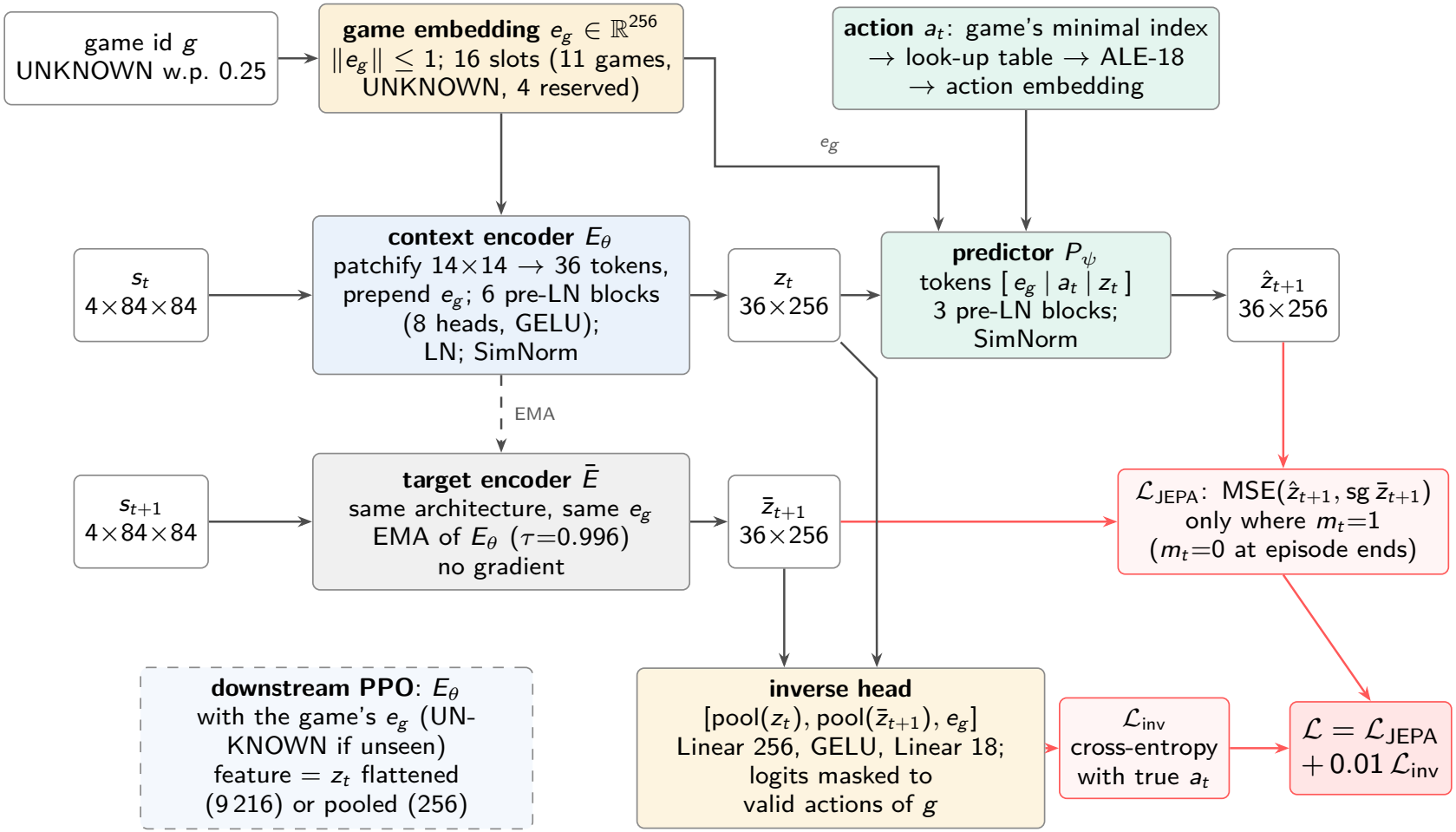


round to the nearest level

4 channels with $L = [8, 5, 5, 3]$ levels; no codebook to learn

- Small moving objects got lost, so moving pixels get a higher weight in the loss (FSQ-VAE: 85% of the loss)

# V-JEPA World Model




The model predicts the *latent* of the next frame stack, not its pixels. The target encoder is a slowly updated copy of the context encoder (no gradients). Only the context encoder is used for PPO.

# V-JEPA: Design Choices

- Following TD-MPC2, so that one model can be trained on many games:
  - one common set of 18 actions (the same index is a different button in each game)
  - a learnable embedding per game ("UNKNOWN" 25% of the time)
  - SimNorm: latents in groups of 8 with a softmax, against collapse
  - a head that guesses the action: $\mathcal{L} = \mathcal{L}_{\text{JEPA}} + 0.01\,\mathcal{L}_{\text{inv}}$

| Part | Params | learns from $\mathcal{L}_{\text{JEPA}}$ | from $\mathcal{L}_{\text{inv}}$ |
|---|---|---|---|
| Context encoder | 4.95 M | yes | yes |
| Predictor | 2.37 M | yes | no |
| Inverse head | 0.20 M | no | yes |
| Target encoder | 4.95 M | moving average only | |

# Experimental Setup: Pre-Training

| | Std. VAE | Res. VQ-VAE | FSQ-VAE | V-JEPA |
|---|---|---|---|---|
| Data | 550 k stacks | 550 k stacks | 550 k stacks | 2.75 M transitions |
| Length | 58 epochs | 29 epochs | 125 epochs | 92 epochs |
| Steps | 250 k | 250 k | 537 k | 494 k |
| Batch | 128 | 64 | 128 | 512 |
| Optimiser, LR | Adam, $10^{-3}$ | $3{\cdot}10^{-4}$ | AdamW, $3{\cdot}10^{-4}$ | AdamW, $3{\cdot}10^{-4}$ |
| Parameters | 2.56 M | 0.70 M | 0.84 M | 12.5 M |
| Hardware, time | H100, 0.8 h | H100, 0.9 h | H100, 1.9 h | H100, 15.6 h |

- The architectures follow the original papers (VQ-VAE, FSQ, I-JEPA, TD-MPC2)
- The JEPA papers train huge models on many GPUs, so a smaller batch (512) and a lower learning rate were used for the 12.5 M model on one GPU
- For the V-JEPA, the checkpoint with the lowest total validation loss $\mathcal{L}_{\text{JEPA}} + 0.01\,\mathcal{L}_{\text{inv}}$ (epoch 92) was used; a small sweep confirmed the inverse-loss weight $\lambda = 0.01$

# Experimental Setup: PPO and Evaluation

- **Baseline:** NatureCNN trained from scratch
- **Frozen:** pre-trained encoder stays fixed, only the actor and critic learn
- **Fine-tuned:** pre-trained encoder keeps learning together with the policy
- **PPO:** standard RL-Zoo Atari settings, 10 M steps, same 512-unit heads for every encoder
- **Metric:** mean return over the last 1 M steps, turned into the human-normalised score so that games can be compared

$$\text{HNS} = \frac{\text{score} - \text{random}}{\text{human} - \text{random}}$$

- One seed per run; all numbers come from the full Weights & Biases logs

# Scope of the Work

**15**
Atari games
(11 + 4 held out)

**3.8 M**
samples from
expert agents

**5**
encoders
pre-trained

**6**
tests of the
world model

**168**
PPO runs of
10 M steps each

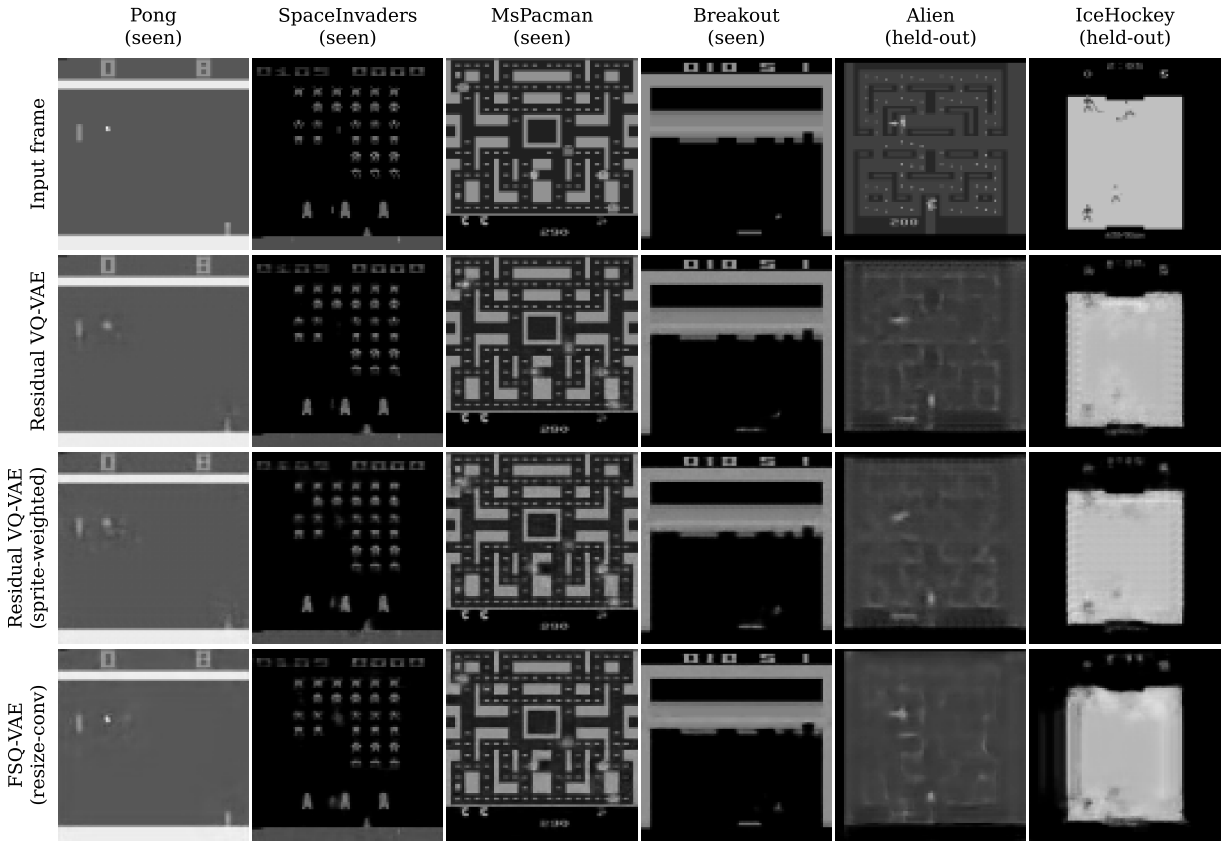
**1.68 B**
agent steps
in total

**≈10 k**
lines of
Python code

PPO runs: NatureCNN 15 · Std-VAE 30 · Res-VQ 30 · FSQ-VAE 30 · V-JEPA 63

May–September 2026, ZIH *Capella* cluster (NVIDIA H100)

# Pre-Training Results




- **FSQ-VAE:** lowest error, and 3× less error on moving pixels than the VQ-VAE
- **Held-out games:** all autoencoders are much worse; they fit the training games
- **V-JEPA:** the loss went *up* after epoch 2 while the latents became richer (effective rank 1.4 → 57); the low early loss came from almost collapsed latents

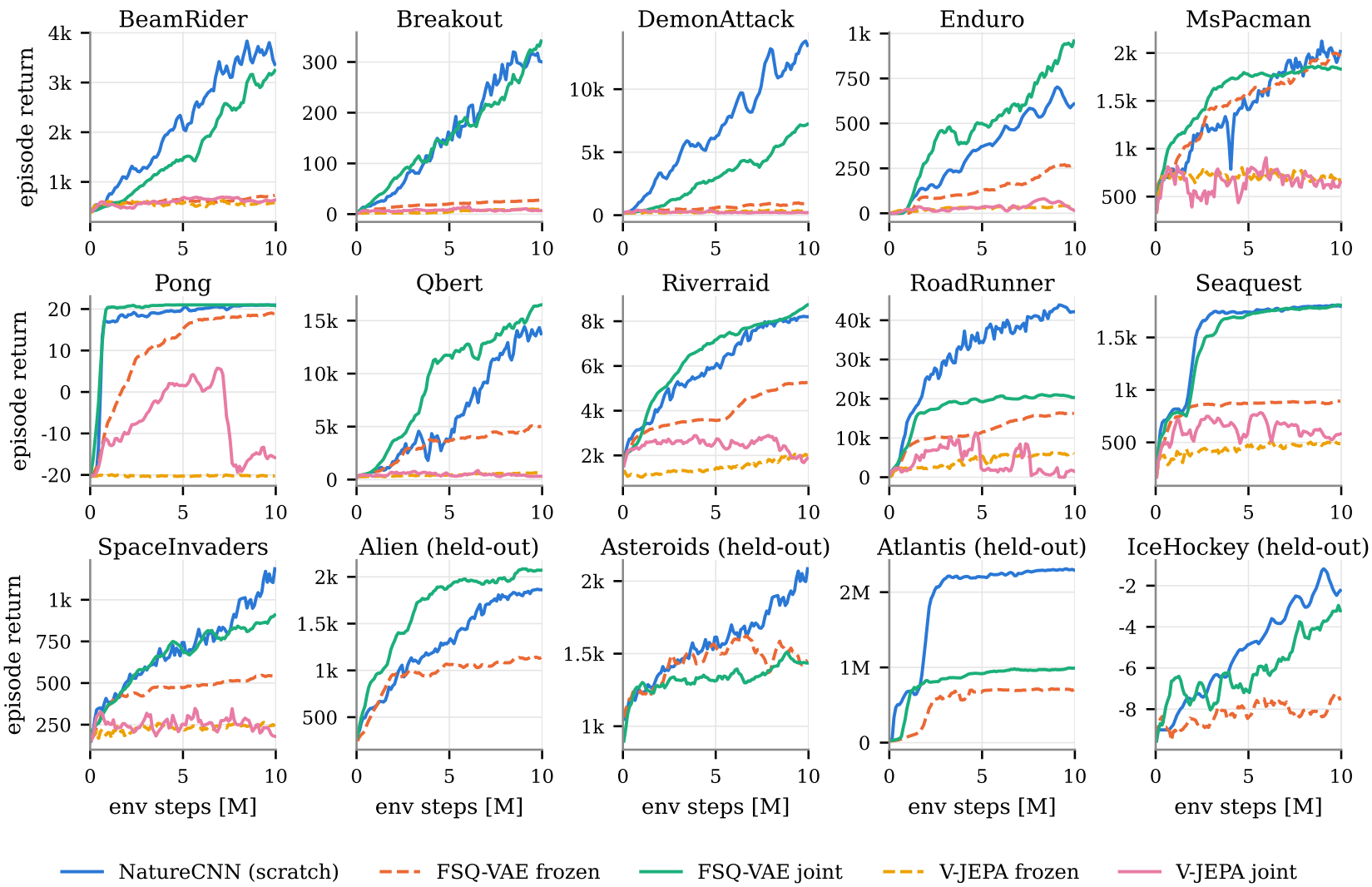
# Downstream RL Performance

| | NatureCNN | Std-VAE | | Res-VQ | | FSQ-VAE | | V-JEPA | |
|---|---|---|---|---|---|---|---|---|---|
| | scratch | frzn | joint | frzn | joint | frzn | joint | frzn | joint |
| Med HNS (seen) | 0.74 | 0.06 | 0.36 | 0.17 | 0.85 | 0.31 | **1.07** | 0.05 | 0.04 |
| Mean HNS (seen) | **2.51** | 0.14 | 0.53 | 0.30 | 0.91 | 0.54 | 2.02 | 0.13 | 0.07 |
| Med HNS (held-out) | **0.50** | 0.01 | 0.05 | 0.05 | 0.21 | 0.21 | 0.45 | 0.05 | 0.06 |

Std-VAE, Res-VQ and the NatureCNN/V-JEPA held-out scores were recorded rounded

- **FSQ fine-tuned:** highest median (1.07 vs. 0.74), beat NatureCNN on 4 of 11 games and nearly tied on Pong and Seaquest
- NatureCNN stayed ahead on the other games (about 2× on DemonAttack and RoadRunner), so its mean is higher
- **Other VAEs:** Std-VAE lowest on every game; Res-VQ below FSQ except on MsPacman, furthest behind on small, fast sprites
- **FSQ frozen:** learned something in every game, but reached less than a third of the fine-tuned median
- **FSQ:** fine-tuning beat freezing on 13 of 15 games (Breakout 12×)
- **Held-out games:** NatureCNN ahead of FSQ fine-tuned on 3 of 4
- **V-JEPA:** learned very little, frozen or fine-tuned

# Learning Curves




Solid: trained or fine-tuned encoders; dashed: frozen encoders

# Sample Efficiency

Million steps until an agent first covers 50% / 80% of NatureCNN's learning progress (–: never); fastest in bold

| | 50% level | | | 80% level | | |
|---|---|---|---|---|---|---|
| Game | Nature | FSQ-J | FSQ-F | Nature | FSQ-J | FSQ-F |
| BeamRider | **4.3** | 6.8 | – | **6.8** | 8.6 | – |
| Breakout | **5.0** | 5.2 | – | **7.0** | 8.2 | – |
| DemonAttack | **5.2** | 9.3 | – | **7.5** | – | – |
| Enduro | 4.3 | **2.2** | – | 7.2 | **5.8** | – |
| MsPacman | 2.5 | **1.9** | 2.6 | 6.2 | **3.8** | 7.2 |
| Pong | 0.7 | **0.6** | 1.8 | **0.7** | 0.8 | 4.2 |
| Qbert | 6.0 | **3.5** | – | 7.3 | **4.7** | – |
| Riverraid | 2.9 | **2.5** | 8.8 | 6.2 | **4.8** | – |
| RoadRunner | **1.9** | – | – | **4.0** | – | – |
| Seaquest | **1.9** | 2.0 | – | **2.2** | 3.1 | – |
| SpaceInvaders | **3.6** | 4.0 | – | **8.0** | – | – |
| FSQ joint faster | | 5/11 | | | 4/11 | |

- FSQ fine-tuned (FSQ-J) reached the 80% level 1.4–2.6 M steps earlier on Enduro, MsPacman, Qbert and Riverraid
- FSQ frozen (FSQ-F) reached the 80% level on only 2 of 11 games

# V-JEPA: Can the Features Be Fixed?

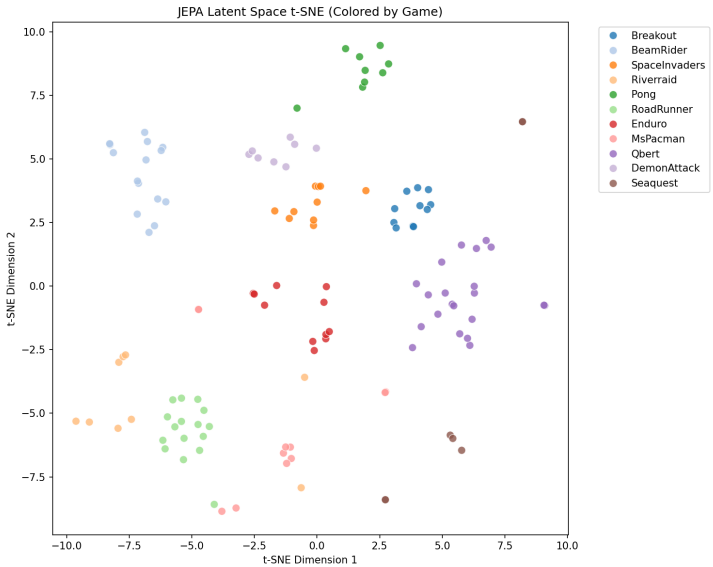
- Hypothesis: the 9 216-dimensional feature is too big for PPO
- All 22 seen-game runs were repeated with the tokens averaged into a **256-dim** feature (these runs also used separate actor and critic encoders)
- The frozen runs were repeated with a 5× larger **entropy bonus**

| Feature | Mode | Median HNS | Critic expl. var. |
|---|---|---|---|
| 9 216 (flat) | frozen | 0.045 | 0.73 |
| 256 (pooled) | frozen | 0.021 | 0.53 |
| 256 (pooled) | fine-tuned | 0.007 | −0.27 |
| 256, entropy 0.05 | frozen | 0.015 | 0.03 |

- Neither change helped, so the size of the feature was probably not the problem, but the information in it

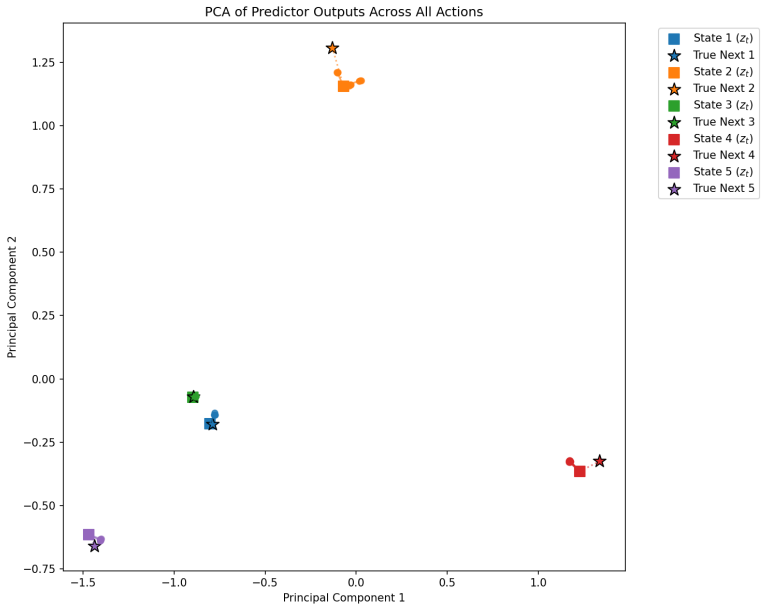
# V-JEPA: World-Model Diagnostics

- Next states share 3 of 4 frames, so **copying** the current latent already looks good. Baselines: copying, an untrained model and **wrong actions**
- **No collapse:** effective rank 52.5 vs. 2.5 untrained
- **It learned dynamics:** better than copying, gap grows from +0.013 (1 step) to +0.046 (5 steps)
- **But the action barely matters:** a wrong action changes the prediction by only 0.006
- The game token gets 1.8× uniform attention, but the encoder already recognises the game from the pixels
- On held-out games it is worse than copying




t-SNE of V-JEPA latents: they group by game

# Why the V-JEPA Fails with PPO




The predictions for all actions (dots) land almost on the same spot

## Observations

- Predictor mostly ignores the action
- Latents barely change per step (cos. 0.974)
- Latent space is sorted by game
- Worse on held-out games

## Interpretation

- JEPA can ignore details with little effect on the latent error, like a small ball
- $14 \times 14$ patches are coarse for small objects
- Expert data shows few other actions
- The critic could read how good a state is, but the policy could not find the right action

# Compute Cost and Energy

| | | | per run | |
|---|---|---|---|---|
| Stage | Model | Runs | GPU-h | kWh |
| Pre-training | Std-VAE / Res-VQ* | 3 | 0.8 / 0.9 | 0.29 / 0.33 |
| | FSQ-VAE | 1 | 1.9 | 0.70 |
| | V-JEPA | 1 | 15.6 | 1.96 |
| PPO (10 M steps) | NatureCNN | 15 | 3.1 | 0.39 |
| | Std-VAE, Res-VQ* | 60 | 2.2 | 0.32 |
| | FSQ-VAE | 30 | 2.2 | 0.31 |
| | V-JEPA frozen / fine-tuned | 30 | 3.6 / 4.5 | 0.55 / 0.84 |
| | V-JEPA ablations | 33 | 4.0 | 0.67 |
| **Total** | reported experiments | 177 | **≈520 GPU-h, 82 kWh** | |

per run: mean over runs; * approximate (not logged in W&B); energy = measured GPU power, one NVIDIA H100 per run

- Pre-training is a **one-off cost** of about one PPO run and is shared by all games
- During PPO the GPU is mostly idle (the Atari emulator runs on the CPU): 125–210 W of 700 W
- **Interpretability:** the autoencoder decoders turn features back into images; the V-JEPA can only be inspected indirectly (PCA, t-SNE, probes)

# Conclusion

- **RQ1:** the FSQ-VAE gave the most useful features when fine-tuned (beat Std-VAE on every game, Res-VQ on all but one)
- **RQ2:** fine-tuning worked much better than freezing
  - FSQ fine-tuned vs. NatureCNN: higher median HNS (1.07 vs. 0.74) and faster learning on 4 games, but a better final score on only 4 of 11
- **RQ3:** no overall gain on unseen games (FSQ better only on Alien, NatureCNN ahead on the other 3)
- **RQ4:** pre-training costs about one PPO run (FSQ-VAE: 1.9 GPU-h, 0.7 kWh) and is shared by all games; all experiments took ≈520 GPU-h and 82 kWh. The FSQ decoder shows what the features contain
- The V-JEPA learned how games usually evolve, but hardly what the agent's actions change
- **Overall:** a shared encoder is a useful starting point, but it cannot yet replace the NatureCNN when frozen

# Limitations and Future Work

- **Limitations:** one seed per run, PPO not re-tuned for the pre-trained encoders
- **Future work:**
  - Repeat the main comparison with 3 seeds
  - V-JEPA with smaller patches and a loss where the action matters more (multi-step prediction, reward head)
  - Check the features first by predicting object positions, before running PPO
  - Use the FSQ grid inside a world model, and pre-train on more games

# Thank You!

Questions?